

Django REST Framework (DRF) - Scenario-Based MCQs (1-25)

1. You have a Book model with a published_date field. You want the API to return a calculated field is_recent (True if published in the last 30 days). Where is the most appropriate place to implement this logic?

- A) Override the create method in the BookViewSet.
- B) Add a method in the BookSerializer using a SerializerMethodField.
- C) Add a property on the Book model.
- D) Perform the calculation in the frontend after fetching the data.

Answer: B

Explanation: While C (a model property) is also valid, SerializerMethodField is the standard DRF way to add read-only, calculated fields to your API representation without persisting them in the database. It keeps the API-specific logic in the serializer.

2. Your Profile model has a OneToOneField to the built-in User model. In your ProfileSerializer, you want to include the user's username and email. How do you achieve this?

- A) Use a nested serializer, e.g., user = UserSerializer(read_only=True).
- B) Use a PrimaryKeyRelatedField pointing to the User model.
- C) Use a SlugRelatedField with slug_field='username'.
- D) Both A and C are valid approaches.

Answer: D

Explanation: A is great for outputting a full nested object. C is more efficient if you only need one field from the related object (like the username). B would only output the user's primary key, not the username or email.

3. You are using a ModelSerializer for a Project model. The model has a status field with choices ('PLANNED', 'ONGOING', 'COMPLETED'). You want the API response to use the choice keys but ensure the API input is validated against these choices. What does the ModelSerializer generate for this field by default?

- A) A CharField with no validation.
- B) A ChoiceField which automatically enforces validation and uses the choice keys for representation.
- C) You must manually define a ChoiceField in the serializer to get validation.
- D) An IntegerField if the choices are stored as integers in the database.

Answer: B

Explanation: ModelSerializer is smart enough to inspect the model field and automatically generate a ChoiceField with the correct choices, handling both representation and validation.

4. A client sends a POST request to create a new object, but the payload includes a read-only field (e.g., created_by which should be set server-side). What is the default behavior of a DRF ModelSerializer?

- A) It will ignore the read-only field in the input and create the object.
- B) It will raise a validation error because the read-only field was included in the input.
- C) It will use the client's value for the read-only field, creating a security issue.
- D) It will set the field to None.

Answer: A

Explanation: Read-only fields are excluded from validation and the validated_data dictionary. The serializer simply ignores them during create/update, which is the secure and desired behavior.

5. You have an Article model with an ImageField. Your ArticleSerializer uses a ModelSerializer. When you serialize an article, you want the image field to return absolute URLs (e.g., https://example.com/media/cat.jpg). How do you achieve this?

- A) Manually construct the URL in the view before sending the response.
- B) Use the context argument when initializing the serializer: serializer = ArticleSerializer(article, context={'request': request}).
- C) Override the to_representation method of the serializer and use request.build_absolute_uri(image.url).
- D) Both B and C are correct. Using the context is the standard way.

Answer: D

Explanation: B is the standard and easiest method. DRF's ImageField and FileField will automatically build absolute URLs if a request object is passed in the context. C is a more manual approach that achieves the same result.

6. You need an API endpoint that performs a non-CRUD operation: /api/products/{id}/restock/. You are using a ModelViewSet for the Product model. What is the best way to implement this?

- A) Create a separate function-based view and add it to the URL configuration.
- B) Use a @action decorator on a method within the ProductViewSet.

- C) Override the update method in the ProductViewSet to handle the restock logic.
- D) This is not possible with a ViewSet; you must use a generic APIView.

Answer: B

Explanation: The `@action` decorator is specifically designed for this purpose. It allows you to add custom endpoints that map to custom methods on your ViewSet, keeping all related logic together. `detail=True` makes it a per-instance action.

7. You have a ViewSet that only needs to support list() and retrieve() actions. What is the most efficient base class to use?

- A) ViewSet
- B) ModelViewSet
- C) ReadOnlyModelViewSet
- D) GenericAPIView

Answer: C

Explanation: `ReadOnlyModelViewSet` provides only the list and retrieve actions, making it more explicit and secure than using the full `ModelViewSet` and disabling actions.

8. You are using a ModelViewSet with a router. By default, the router generates the URL pattern for retrieving a single object as '^products/{pk}/\$'. You need to change this to '^products/{id}/\$'. How do you do it?

- A) Modify the `lookup_field` attribute on the ViewSet to 'id'.
- B) Modify the `lookup_url_kwarg` attribute on the ViewSet to 'id'.
- C) This is configured in the router: `router = SimpleRouter(trailing_slash=False, lookup_url_kwarg='id')`.
- D) You cannot change it; `pk` is hardcoded.

Answer: B

Explanation: `lookup_url_kwarg` controls the name of the keyword argument in the URL pattern. `lookup_field` controls the name of the field on the model used for the lookup. They are often the same (e.g., `pk`), but can be different. To change the URL conf, you change `lookup_url_kwarg`.

9. In a class-based APIView, you have defined a custom post method. However, you also want to run some specific logic every time a request is made, regardless of the HTTP method (e.g., to log the request). Where should you put this logic?

- A) In the post method itself.
- B) In the dispatch method of the view.

- C) In a custom middleware.
- D) In the `perform_authentication` method.

Answer: B

Explanation: The `dispatch` method is the first method called for every request. It's the perfect place to put logic that must run for all HTTP methods. Middleware (C) is for logic that runs across the entire project, not just a single view.

10. You have a custom `get_queryset` method in your `ViewSet` that filters objects based on the user's permissions. Which action typically requires a single object and might bypass the `get_queryset` filter if not careful?

- A) `list`
- B) `create`
- C) `retrieve`
- D) `update` and `destroy` (They use `get_object()` which uses `get_queryset()`)

Answer: D

Explanation: While `retrieve` also uses `get_object()`, the key point is that `get_object()` relies on `get_queryset()`. If your `get_queryset` filters based on the user, then `get_object()` will correctly apply those filters before fetching the single object, preventing users from accessing objects they shouldn't. This is a core security feature.

11. Your application uses Token Authentication. A user loses their device. What is the correct procedure to invalidate the existing token and generate a new one?

- A) Delete the Token object from the database; a new one will be created on the user's next login.
- B) Call `user.auth_token.delete()` and then `Token.objects.create(user=user)`.
- C) Set `user.is_active = False` and then back to `True`.
- D) There is no way to invalidate a single token; you must change the user's password.

Answer: A

Explanation: The default DRF Token authentication does not automatically create tokens. It only creates one when you call `Token.objects.create(user=user)`. Therefore, deleting the old token and creating a new one (A) is the manual process. For better security, consider using libraries like `django-rest-auth` or `django-rest-knox` which handle token management better.

Clarification: The most accurate answer based on standard DRF `TokenAuthentication` is to delete the token object. The `auth` endpoint will create a new one on next login. For immediate creation, B is also valid. The best practice is to use a more advanced token management system.

12. You want to configure your API to support both SessionAuthentication (for browsable API usage) and TokenAuthentication (for programmatic clients). How do you set the DEFAULT_AUTHENTICATION_CLASSES?

- A) [SessionAuthentication, TokenAuthentication]
- B) [TokenAuthentication, SessionAuthentication]
- C) It doesn't matter; the client can choose which one to use via the Accept header.
- D) You can only have one default authentication class.

Answer: B

Explanation: Order matters. DRF will try authenticators in the order they are defined.

If SessionAuthentication is first, a logged-in user in the browser will be authenticated via sessions, which is desired for the browsable API. TokenAuthentication will still be available for programmatic calls. Putting Token first would mean every browsable API request would first try (and fail) to find a token header.

13. A client successfully obtains a token using the /auth/login/ endpoint. How should they use this token in subsequent requests to authenticated endpoints?

- A) Add it as a query parameter: ?token=abc123.
- B) Include it in the request body as JSON: {"token": "abc123"}.
- C) Include it in the Authorization header: Authorization: Token abc123.
- D) Include it in a custom header: X-API-TOKEN: abc123.

Answer: C

Explanation: This is the standard and most secure method defined by the TokenAuthentication class. It follows a common HTTP authentication pattern. Query parameters (A) are less secure as they can be logged.

14. You need to implement a custom authentication scheme based on a API key sent in a custom header X-API-KEY. What is the minimum required method to implement in a custom authentication class?

- A) authenticate_header(self, request)
- B) authenticate(self, request)
- C) get_user(self, token)
- D) authenticate_credentials(self, key)

Answer: B

Explanation: The authenticate method is the core method. It should inspect the request for the X-API-KEY header, validate it, and return a (user, auth) tuple if successful, or None if not.

15. For a specific view, you want to allow unauthenticated read-only access (GET, HEAD, OPTIONS) but require authentication for any write operations (POST, PUT, PATCH, DELETE). What is the simplest way to achieve this?

- A) Use the `IsAuthenticatedOrReadOnly` permission class.
- B) Write a custom permission class that checks `request.method`.
- C) Override the `get_authenticators` method to return an empty list for safe methods.
- D) Use `AllowAny` for GET and `IsAuthenticated` for others in `permission_classes`.

Answer: A

Explanation: `IsAuthenticatedOrReadOnly` is a built-in permission class that does exactly this. It's the simplest and most standard solution.

16. You have a Project model with an owner field (a FK to User). You want to ensure that only the owner of a project can update or delete it. What is the best permission class to use?

- A) `IsAuthenticated`
- B) `IsAdminUser`
- C) `DjangoModelPermissions`
- D) A custom permission class that checks `obj.owner == request.user`.

Answer: D

Explanation: The built-in permissions cannot handle object-level logic like ownership. You must write a custom permission class that implements `has_object_permission(self, request, view, obj)` and checks the relationship between the `obj` and the `request.user`.

17. You are using DjangoModelPermissions. A GET request is made by an authenticated user to a view. What permission does the user need on the underlying model?

- A) The view permission (e.g., `app.view_modelname`).
- B) The change permission.
- C) No permission is needed for GET requests.
- D) The add permission.

Answer: A

Explanation: `DjangoModelPermissions` requires users to be authenticated and checks for model-level permissions. For GET requests, it checks for the view permission. This is a change from older Django versions and requires you to add view permissions to your models.

18. You want to create a custom permission class `IsProjectMember` that grants access if the user is in the members `ManyToManyField` of a Project object. For which object-level permission method should you implement the logic?

- A) `has_permission(self, request, view)`
- B) `has_object_permission(self, request, view, obj)`
- C) `has_membership(self, request, obj)`
- D) `check_permission(self, request, obj)`

Answer: B

Explanation: `has_object_permission` is used for permissions that require a specific object instance to check against, which is exactly what you need to check if the user is in `obj.members`.

19. The `has_object_permission` method is not called for a list action in a `ViewSet`. Why?

- A) This is incorrect; it is called for every object in the queryset during list.
- B) Because there is no single object to check against in a list action.
- C) You need to set `check_object_permissions = True` on the `ViewSet`.
- D) The list action only uses `has_permission`.

Answer: D

Explanation: Object-level permissions (`has_object_permission`) are only checked when a single object is retrieved, updated, or deleted. For the list action, which returns a queryset, only the view-level permission (`has_permission`) is checked. It's the developer's responsibility to filter the queryset in `get_queryset` based on the user's permissions.

20. You have a view where you want to allow anyone to create a new object (POST), but only the object's owner can later edit it. How would you structure your permissions?

- A) Use `AllowAny` for create and a custom `IsOwner` permission for update.
- B) Override the `get_permissions` method in the view to use different permission classes for different actions.
- C) Use `IsAuthenticatedOrReadOnly` and let the update logic handle ownership in the serializer.
- D) This is not possible; you must use the same permission class for all actions in a view.

Answer: B

Explanation: This is a common pattern. You can override `get_permissions` to return different lists of permission classes. For example:

python

```
def get_permissions(self):
    if self.action == 'create':
        return [AllowAny()]
    else:
```

```
return [IsAuthenticated(), IsOwner()]
```

21. A client sends a PATCH request with a JSON payload that contains a field not defined in the serializer. What is the default behavior?

- A) The serializer ignores the extra field and proceeds with validation.
- B) The serializer raises a validation error immediately.
- C) The serializer includes the extra field in the validated_data dict.
- D) The behavior is controlled by the partial parameter.

Answer: A

Explanation: By default, DRF serializers ignore extra fields. To raise an error for extra fields, you need to set `extra_kwargs = {'extra_field_name': {'write_only': True}}` or more commonly, use the strict validation option in newer versions or a custom validation step.

Clarification: In standard practice, extra fields are ignored. The ability to raise an error is a configuration option.

22. You need to make the email field on your User serializer required only during creation (POST), but not during updates (PATCH). How can you achieve this?

- A) This is not possible; field requirements are fixed in the serializer.
- B) Override the `__init__` method of the serializer and set `self.fields['email'].required = False` if the instance exists.
- C) Use two different serializers: one for creation and one for update.
- D) Both B and C are valid strategies.

Answer: D

Explanation: B is a common dynamic approach. C is often cleaner and more explicit, especially if the creation and update logic diverge significantly. You can override `get_serializer_class` in your `ViewSet` to choose the appropriate serializer based on the action.

23. You want to change the format of datetime fields in your API responses to use Unix timestamps instead of ISO 8601 strings. What is the best approach?

- A) Manually convert them in the view before returning the response.
- B) Define a custom serializer field class, e.g., `UnixTimestampField`, and use it in your serializer.
- C) Override the `to_representation` method for each datetime field in the serializer.
- D) Use a custom renderer.

Answer: B

Explanation: Creating a custom field class (B) is the most reusable and DRY approach. You can

then use this field anywhere in your project. C would work but is less reusable. A is not maintainable. D would change the format for the entire API, not just datetime fields.

24. Where should you put business logic that must be executed every time a new object is created, such as sending a welcome email or creating related objects?

- A) In the create method of the View.
- B) In the create method of the Serializer.
- C) In the save method of the Model.
- D) In a post-save signal.

Answer: B

Explanation: The serializer's create (and update) method is the recommended place for logic that is directly tied to the process of creating/updating an object from validated data. Signals (D) can be used but can make code harder to follow and test.

25. You want to add a custom validation rule to a serializer field: the username field must not contain the word "admin". Where should you implement this?

- A) In the validate_username method of the serializer.
- B) In the clean_username method of the model.
- C) In a validate method of the serializer that checks all fields.
- D) A or C are both correct, but A is more specific and cleaner.

Answer: D

Explanation: Field-specific validation should be placed in validate_<field_name> methods (A). The general validate method (C) is for validation that requires multiple fields. A is the cleaner and more specific choice for this scenario.

Django REST Framework (DRF) - Scenario-Based MCQs (26-50)

26. You are using PageNumberPagination. A client wants to request the third page of results, with a page size of 25. What would the typical request URL look like?

- A) /api/items/?page=3&size=25
- B) /api/items/?page=3&page_size=25
- C) /api/items/?p=3&page_size=25
- D) /api/items/?page=3&per_page=25

Answer: B

Explanation: The default query parameters for PageNumberPagination are page for the page number and page_size to override the default page size.

27. You need to create a custom pagination class that returns a response with a different structure, e.g., {'data': results, 'meta': {'page': 3, 'pages': 10}}. Which method should you override?

- A) `paginate_queryset(self, queryset, request, view=None)`
- B) `get_paginated_response(self, data)`
- C) `get_paginated_response_schema(self, schema)`
- D) `get_page_size(self, request)`

Answer: B

Explanation: The `get_paginated_response` method takes the serialized page data and constructs the final response dictionary. This is where you define the custom structure.

28. You are using `LimitOffsetPagination`. A client makes a request to `/api/items/?limit=10&offset=20`. What are they asking for?

- A) The first 10 items.
- B) The 10 items starting from the 20th item (i.e., items 21-30).
- C) 10 items from the end of the list.
- D) The 20th item, and 10 items before it.

Answer: B

Explanation: `offset` specifies the starting position (0-indexed) of the first item to return. `limit` specifies the maximum number of items to return. `offset=20, limit=10` means "skip the first 20 items, then give me up to 10."

29. You want to add filtering to your `ViewSet` so clients can filter products by category and price. Which backend should you use?

- A) `SearchFilter`
- B) `OrderingFilter`
- C) `DjangoFilterBackend`
- D) A custom filter backend.

Answer: C

Explanation: `DjangoFilterBackend`, especially with a `FilterSet`, is designed for exact field lookups like this. `SearchFilter` is for broad, multi-field text search, and `OrderingFilter` is for sorting.

30. A client wants to search for products where the name or description contains the word "organic". Which filter backend is most suitable?

- A) DjangoFilterBackend
- B) SearchFilter
- C) OrderingFilter
- D) A custom filter.

Answer: B

Explanation: The SearchFilter backend is specifically designed for this type of broad, multi-field search. You configure the searchable fields with `search_fields = ['name', 'description']`.

31. You need to implement complex filtering, e.g., products with a price between 10 and 100. How can you best achieve this with django-filter?

- A) This is not possible with django-filter.
- B) Use the SearchFilter and a complex query string: `?search=price:10-100`.
- C) Define a ProductFilter FilterSet and use the RangeFilter for the price field.
- D) Handle it manually in `get_queryset` by parsing `request.GET`.

Answer: C

Explanation: The django-filter library provides powerful filters like RangeFilter, NumberFilter, and ChoiceFilter which allow you to easily build complex query interfaces without manual parsing.

32. You want to allow clients to order results by name or -created_at (newest first). Which backend should you use?

- A) DjangoFilterBackend
- B) SearchFilter
- C) OrderingFilter
- D) A custom filter.

Answer: C

Explanation: The OrderingFilter backend is built for this purpose. You specify `ordering_fields = ['name', 'created_at']`, and the client can use `?ordering=name` or `?ordering=-created_at`.

33. You have implemented a simple rate limit of 100 requests per day for an API endpoint using SimpleRateThrottle. A user makes 100 requests in the first hour. What happens on their 101st request within the same day?

- A) The request is processed normally.
- B) The request is processed, but a warning is added to the response headers.

- C) The request is throttled and returns a 429 Too Many Requests response.
- D) The request is throttled and returns a 403 Forbidden response.

Answer: C

Explanation: This is the core function of throttling. Once the rate limit is exceeded, all subsequent requests within the defined time period are throttled with a 429 status code.

34. You need to apply a stricter throttle rate for anonymous users (10/hour) than for authenticated users (100/hour). How would you structure this?

- A) Use AnonRateThrottle and UserRateThrottle in your DEFAULT_THROTTLE_CLASSES and set DEFAULT_THROTTLE_RATES accordingly.
- B) Write a single custom throttle class that checks request.user.is_authenticated.
- C) It's not possible to have different rates for anonymous and authenticated users.
- D) Use ScopedRateThrottle for this purpose.

Answer: A

Explanation: This is the exact purpose of the built-in AnonRateThrottle and UserRateThrottle classes. You can define their rates in the DEFAULT_THROTTLE_RATES setting, e.g., {'anon': '10/hour', 'user': '100/hour'}.

35. You want to apply a throttle specifically to the login endpoint to prevent brute-force attacks. Which throttle class is best suited?

- A) UserRateThrottle
- B) AnonRateThrottle
- C) ScopedRateThrottle
- D) A custom throttle.

Answer: C

Explanation: ScopedRateThrottle allows you to define specific throttle rates for different views by setting a throttle_scope attribute on the view. You can have a very restrictive scope for your login view (e.g., '5/minute') and different scopes for other parts of your API.

36. You are writing a test for an API endpoint. What is the main advantage of using APITestCase over Django's standard TestCase?

- A) APITestCase runs tests faster.
- B) APITestCase provides the APIClient for making test requests.
- C) APITestCase automatically migrates the test database.
- D) There is no significant difference.

Answer: B

Explanation: APITestCase extends Django's TestCase and provides an APIClient class, which is much easier to use for testing REST APIs than Django's standard client. It handles encoding JSON, setting headers, and parsing JSON responses automatically.

37. In a test, you need to authenticate a user before making a request. How do you do this with the APIClient?

- A) `client.login(username='test', password='test')`
- B) `client.force_authenticate(user=user)`
- C) `client.credentials(HTTP_AUTHORIZATION='Token ' + user.auth_token.key)`
- D) Both B and C are correct.

Answer: D

Explanation: B (`force_authenticate`) is the quickest method for tests, as it bypasses the actual authentication mechanism. C is more realistic as it simulates the actual Authorization header a client would send. A uses session authentication, which may not be what you want for an API.

38. You notice your API is making hundreds of database queries when listing objects with related foreign keys (N+1 problem). How do you solve this in DRF?

- A) Use the `select_related` and `prefetch_related` methods in the ViewSet's `get_queryset` method.
- B) Write raw SQL to optimize the query.
- C) Use a faster database.
- D) This is not a problem; it's expected behavior.

Answer: A

Explanation: This is a classic ORM optimization. `select_related` is for foreign keys and one-to-one relationships (fetches them in a single SQL JOIN). `prefetch_related` is for many-to-many and reverse foreign key relationships (fetches them in a separate query and does the 'joining' in Python).

39. You have a deeply nested serializer that is causing performance issues. What is a better alternative?

- A) Use a flat serializer that only uses primary keys or slugs for related objects.
- B) Remove the nested serializer and handle relations manually in the view.
- C) Use a library like `drf-flex-fields` to dynamically control the nesting depth.
- D) All of the above are valid strategies.

Answer: D

Explanation: Deep nesting is a common performance anti-pattern. All these are solutions: A

(flattening), B (manual control), and C (a popular library that allows clients to specify which fields to expand using `?expand=field1.field2`).

40. You want to create a ViewSet action that does NOT get routed by the default router (e.g., it's a utility method for internal use). How do you prevent this?

- A) Start the method name with an underscore `_`.
- B) Set `detail=None` on the `@action` decorator.
- C) Pass `routed=False` to the `@action` decorator.
- D) The `@action` decorator only routes methods if `url_path` is explicitly set.

Answer: A

Explanation: The default SimpleRouter will only generate routes for methods that are explicitly decorated with `@action` or are part of the standard set (list, create, etc.). It will ignore any method whose name starts with an underscore. This is the convention to mark an action as private.

41. You need to version your API. You want clients to specify the version in the URL (e.g., `/api/v1/products/`). What is the best approach?

- A) Use the `URLPathVersioning` scheme.
- B) Use the `NamespaceVersioning` scheme.
- C) Use the `AcceptHeaderVersioning` scheme.
- D) Manually parse the version from `request.path` in every view.

Answer: A

Explanation: `URLPathVersioning` is designed for including the version as part of the URL path, which is a common and clear pattern. It works seamlessly with DRF's routers.

42. You have a legacy database table with a non-standard primary key field named `uid`. How do you make your DRF ModelViewSet use this field for object lookups?

- A) Set `lookup_field = 'uid'` on the ViewSet.
- B) Set `lookup_url_kwarg = 'uid'` on the ViewSet.
- C) Set `pk_field = 'uid'` on the model.
- D) Both A and B are required.

Answer: D

Explanation: `lookup_field` tells the ViewSet which model field to use for the lookup ('uid'). `lookup_url_kwarg` tells the ViewSet what name to expect in the URL pattern (e.g., `/api/items/{uid}/`). You typically need to set both.

43. You want to add a field to your serializer that is a hyperlink to a related object's detail endpoint, but you don't want to use the default HyperlinkedModelSerializer behavior. Which field should you use?

- A) HyperlinkedIdentityField
- B) HyperlinkedRelatedField
- C) SerializerMethodField and manually build the URL.
- D) URLField

Answer: A

Explanation: HyperlinkedIdentityField is used to generate a hyperlink to the object being serialized (its own detail page). HyperlinkedRelatedField is for generating links to related objects.

44. During deserialization, you need to create multiple related objects based on the input data. Where is the best place to put this logic?

- A) In the create method of the View.
- B) In the create method of the Serializer.
- C) In the perform_create method of the View.
- D) In a post-save signal.

Answer: B

Explanation: The serializer's create and update methods are responsible for turning validated data into model instances. This is the natural place to handle the creation of related objects using, for example, serializers.PrimaryKeyRelatedField or by manually processing nested data.

45. You want to add a non-model field (e.g., current_time) to your serializer output. How do you define this field?

- A) Add it to the model as a property.
- B) Define it as a CharField(read_only=True) in the serializer and populate it in the view.
- C) Define it as a SerializerMethodField in the serializer and define a get_current_time method.
- D) Use a JSONField and include it in the data.

Answer: C

Explanation: SerializerMethodField is the standard way to add read-only, calculated data to your serialized output. The method get_<field_name> defines how to calculate the value.

46. A client's POST request fails validation. What HTTP status code does DRF return by default?

- A) 200 OK
- B) 400 Bad Request
- C) 403 Forbidden
- D) 500 Internal Server Error

Answer: B

Explanation: A 400 Bad Request is the correct status code for a request that contains invalid data and cannot be processed.

47. You need to return a custom error response format from your API, e.g., { "error": { "code": "invalid_email", "message": "Email is invalid." } }. How can you achieve this globally?

- A) Override the `handle_exception` method in your custom API view base class.
- B) Change the `REST_FRAMEWORK['EXCEPTION_HANDLER']` setting.
- C) Use a custom renderer.
- D) Both A and B are valid.

Answer: D

Explanation: You can override exception handling at the view level (A) or, more commonly, at the global project level (B) by defining a custom exception handler function. This function can catch any API exception and format the response as desired.

48. You are using Django's `get_user_model()` function in your `models.py`. What is a potential issue, and how is it avoided in DRF serializers?

- A) It can cause circular imports. DRF provides a `user_model` import shortcut.
- B) It is slower. DRF uses a faster method.
- C) It doesn't work with custom user models. DRF automatically detects the user model.
- D) There is no issue.

Answer: A

Explanation: Referencing the user model in `models.py` is fine. However, importing `models.py` into `serializers.py` and vice-versa can cause circular imports. The standard solution, which DRF also uses in its serializers, is to use a string reference: `serializers.ForeignKey('auth.User')` or more robustly, `settings.AUTH_USER_MODEL`.

49. You want to create a BulkDelete action in your ViewSet. What is a key consideration?

- A) It's not possible to do bulk operations with ViewSets.

- B) You must override the router configuration.
- C) You need to handle the fact that the DRF serializer is designed for single objects, not lists.
- D) You must use a APIView for bulk operations.

Answer: C

Explanation: The standard ModelSerializer expects to validate and save a single object. For bulk operations, you often need to write custom logic in your view action to parse a list of IDs, validate permissions for each, and then delete them, bypassing the standard serializer flow for the bulk action itself.

50. You are building a reporting endpoint that returns aggregated data (e.g., sales per category) rather than a list of model instances. What is the best type of view to use?

- A) ModelViewSet
- B) ReadOnlyModelViewSet
- C) A custom APIView or GenericAPIView
- D) A ViewSet with a custom @action

Answer: C

Explanation: When you are not operating on a standard model queryset, a simple APIView is often the clearest and most flexible choice. You can implement get (or other methods) to perform your custom aggregation logic and return a custom response without the overhead of a ModelViewSet.

Django REST Framework (DRF) - Scenario-Based MCQs (51-75)

51. You are creating an object that has a foreign key relationship. The client should only provide the primary key of the related object in the request. Which serializer field is most appropriate?

- A) PrimaryKeyRelatedField
- B) StringRelatedField
- C) SlugRelatedField
- D) A nested serializer

Answer: A

Explanation: PrimaryKeyRelatedField is designed for this exact purpose: to represent the target of the relationship by its primary key during input and output. It validates that the provided PK corresponds to a valid existing object.

52. You need to ensure that a particular view runs inside an atomic transaction, so that all database operations within the view either completely succeed or completely fail. How do you achieve this in a APIView?

- A) Use the `@transaction.atomic` decorator on the HTTP method (e.g., `post`).
- B) Use the `@transaction.atomic` decorator on the entire view class.
- C) It's not possible; transactions must be managed in the serializer.
- D) Django handles this automatically for all views.

Answer: A

Explanation: You can decorate the specific method where the transaction is critical, like `post` or `put`. Decorating the class would apply the transaction to every method (like `get`), which is usually unnecessary and harmful for performance.

53. You have a Project model and a Task model with a FK to Project. You want an endpoint like `/api/projects/123/tasks/` to list all tasks for project 123. What's the best way to implement this?

- A) Create a separate `TaskViewSet` and filter its queryset based on the `project_id` from the URL.
- B) Use a nested router from the `drf-nested-routers` package.
- C) Add a custom `@action(detail=True)` to the `ProjectViewSet` like `@action(detail=True, methods=['get'], url_path='tasks')`.
- D) Any of the above are valid, but B is the most standard for deep nesting.

Answer: D

Explanation: All are valid patterns. A is simple and explicit. B is a common third-party solution for complex nested relationships. C is a good built-in DRF approach that creates an endpoint like `/projects/123/tasks/`. The "best" way depends on the complexity of your needs, but B is often chosen for standard nested resource URLs.

54. You want to add custom metadata to the schema generated by DRF's schema auto-generation for OpenAPI/Swagger. How do you do it?

- A) Modify the generated schema manually.
- B) Create a custom schema generator class and override `get_schema`.
- C) Use the `@swagger_auto_schema` decorator from `drf-yasg`.
- D) This is not possible without writing the entire schema by hand.

Answer: B

Explanation: DRF provides a pluggable schema generation system. You can create a custom class that inherits from `rest_framework.schemas.openapi.AutoSchema` and override its methods (like `get_operation`) to add custom metadata, then specify it on a view with `schema = MyCustomSchema()`.

55. You are implementing a POST endpoint for file uploads. The client will send multipart/form-data. How should your serializer handle the file field?

- A) Use a FileField in the serializer. DRF will handle it automatically.
- B) You cannot use a serializer; you must handle request.FILES directly in the view.
- C) Use a JSONField and encode the file as base64.
- D) Use a CharField and expect a URL to the file.

Answer: A

Explanation: DRF's FileField and ImageField are designed to handle file uploads. When the view receives a multipart request, the serializer will correctly bind the uploaded file from request.FILES to the field for validation and saving.

56. You need to change the default format of validation errors from {"field_name": ["Error message."]} to {"errors": [{"field": "field_name", "message": "Error message."}]} . Where is the best place to implement this change?

- A) Override the to_internal_value method in every serializer.
- B) Override the raise_error method in the APIView.
- C) Create a custom exception handler function and set it as REST_FRAMEWORK['EXCEPTION_HANDLER'].
- D) This change is not possible.

Answer: C

Explanation: The global exception handler is responsible for converting exceptions (including ValidationError) into responses. By creating a custom handler, you can catch the ValidationError, reformat its structure, and return your desired response format for every validation error in the project.

57. Your get_queryset method is complex and involves multiple filters based on user permissions and request parameters. How can you effectively test that it returns the correct queryset?

- A) Only test the final API output; testing the queryset directly is redundant.
- B) Test the view using APITestCase and assert the response data length.
- C) Use Django's RequestFactory to create a mock request, call get_queryset(view) and assert on the returned queryset.
- D) It's not possible to unit test get_queryset in isolation.

Answer: C

Explanation: RequestFactory allows you to create a mock request object with specific attributes

(user, GET parameters). You can instantiate your view, set the request attribute, and call `get_queryset()` directly. This lets you unit test the `queryset` logic in isolation from the rest of the view machinery, making tests faster and more precise.

58. You have a `UserProfile` model with a `OneToOne` link to `User`. You want a single API endpoint where a client can update both the user's email and the profile's `phone_number` in one request. How do you structure the serializer?

- A) Create a `UserProfileSerializer` that uses a nested `UserSerializer` with writeable fields.
- B) Tell the client to make two separate `PATCH` requests.
- C) Create a custom serializer that manually handles the validation and saving for both objects.
- D) Both A and C are valid, but C offers more control.

Answer: D

Explanation: A is a good approach if the nested writing behavior of DRF meets your needs. However, for complex relationships or custom logic, C is often necessary. You would override the `create` and `update` methods in the custom serializer to manually update the `User` and `UserProfile` instances based on the validated data.

59. You are using JWT authentication with `django-rest-framework-simplejwt`. The client's access token is about to expire. What should the client do?

- A) Call the login endpoint again with their username and password.
- B) Call the token refresh endpoint with the valid refresh token to get a new access token.
- C) The client must log out and log back in.
- D) There is no way to refresh a JWT; the user must wait for the token to expire and get a new one.

Answer: B

Explanation: A key feature of JWTs is the refresh token workflow. The client stores both an access token (short-lived) and a refresh token (long-lived). Before the access token expires, the client can `POST` the refresh token to the `/token/refresh/` endpoint to obtain a new access token without requiring the user's credentials again.

60. You want to create a "download as CSV" action for a `ViewSet`. What is the best approach?

- A) Use a custom renderer class that renders the data to CSV format.
- B) Generate the CSV in the view and return a `HttpResponse` with the `Content-Type` set to `text/csv`.

- C) Use a SerializerMethodField to format each row as CSV.
- D) This is not possible with DRF.

Answer: A

Explanation: DRF's renderer architecture is perfect for this. You can create a CSVRenderer class that inherits from BaseRenderer, set its media_type to 'text/csv', and implement the render method to convert data into CSV. Then, clients can request CSV by sending an Accept: text/csv header or using a ?format=csv parameter.

61. You need to make an field conditionally required based on the value of another field. For example, if status is "FAILED", then an error_message field must be provided. Where do you implement this?

- A) In the validate_error_message method of the serializer.
- B) In the validate method of the serializer that checks all fields.
- C) In the clean method of the model.
- D) This kind of cross-field validation is not supported.

Answer: B

Explanation: The serializer's validate(self, attrs) method is the place for validation logic that requires access to multiple fields. You can check the value of attrs['status'] and then check for the presence/validity of attrs['error_message'], raising a ValidationError if the condition is not met.

62. You have a ViewSet with a custom @action that uses a different serializer than the default one. How do you handle this?

- A) Override the get_serializer_class method and check for self.action.
- B) Define serializer_class directly on the @action decorator.
- C) You cannot use different serializers for different actions in the same ViewSet.
- D) Instantiate the correct serializer manually inside the action method.

Answer: A

Explanation: The standard pattern is to override get_serializer_class(self). Inside this method, you can check self.action (e.g., if self.action == 'my_custom_action') and return the appropriate serializer class. This keeps the view logic clean.

63. You want to integrate a third-party permission system (e.g., from another service) with your DRF permissions. What is the best way?

- A) Make HTTP calls to the third-party service inside your permission class's has_permission method.
- B) Sync the permission data to your local database and use DjangoModelPermissions.

- C) It's not possible to integrate external systems.
- D) Use a custom middleware for permissions.

Answer: A

Explanation: Permission classes are the correct pluggable component for this. You can create a `ThirdPartyPermission` class that implements `has_permission` and/or `has_object_permission`. Inside these methods, you can make the necessary HTTP calls (preferably using an async-friendly library like `aiohhttp` or caching the results to avoid performance bottlenecks).

64. You notice that a malicious user is trying to overwhelm your login endpoint with rapid requests. What is the most targeted solution?

- A) Apply a very strict global `DEFAULT_THROTTLE_RATES` for all users.
- B) Apply a `ScopedRateThrottle` with a very restrictive rate (e.g., '5/minute') specifically to the login view.
- C) Use `UserRateThrottle` on the login view.
- D) Use `AnonRateThrottle` on the login view.

Answer: D

Explanation: Login attempts are almost always made by anonymous users. Therefore, applying the `AnonRateThrottle` specifically to the login view (by setting its `throttle_scope` or using it in the view's `throttle_classes`) is the most targeted and effective solution to prevent brute-force attacks without affecting authenticated users.

65. You are using `prefetch_related` to optimize a queryset, but you need to also filter the prefetched related objects. How do you do this?

- A) You cannot filter prefetched objects; you must use a separate query.
- B) Use the Prefetch object: `prefetch_related(Prefetch('related_set', queryset=RelatedModel.objects.filter(...)))`.
- C) Filter the list of objects in the serializer after serialization.
- D) Use `select_related` instead.

Answer: B

Explanation: Django's Prefetch object is powerful and allows you to specify a custom queryset for the related objects you are prefetching. This is essential for optimizing queries where you only need a subset of the related objects (e.g., only 'active' comments).

66. A client application needs to check if a user has permission to perform a specific action on a specific object without actually performing the action. What is a RESTful way to provide this?

- A) Create a custom endpoint like POST /api/check-permission/.
- B) Use the OPTIONS HTTP method on the object's endpoint.
- C) Have the client fake the request and parse the response code.
- D) This is not a standard REST practice.

Answer: B

Explanation: The OPTIONS method is intended for this purpose. It returns information about the communication options for the target resource, which can include available HTTP methods and permissions. While DRF's built-in OPTIONS handling is basic, you can override it to include detailed permission information in the response metadata.

67. You want to create a "virtual" or calculated field on your model that is expensive to compute. You only want it computed and included in the API response when explicitly requested by the client. What is a good strategy?

- A) Always compute it; the cost is negligible.
- B) Use a library like drf-flex-fields to make the field optional via query parameters.
- C) Create a separate detailed endpoint that always includes the field.
- D) Use a SerializerMethodField and cache the result of the calculation.

Answer: B

Explanation: This is a perfect use case for dynamic fields. Libraries like drf-flex-fields allow the client to specify which fields to include or exclude (e.g., ?fields=id,name,expensive_field). The serializer then only computes and includes the expensive_field if it was explicitly requested, saving server resources for most requests.

68. You need to run a slow, asynchronous task (e.g., sending an email) after an object is successfully created via the API. What is the best way to trigger this task?

- A) Perform the task synchronously at the end of the serializer's create method.
- B) Use a Django signal and trigger the async task from the signal handler.
- C) Override the perform_create method in the view and use async_to_sync to launch the task.
- D) Use a task queue (e.g., Celery) and trigger the task from the perform_create method or a post-save signal.

Answer: D

Explanation: For slow tasks, you should never block the HTTP response. The standard solution is to use a task queue like Celery. You can trigger the Celery task from perform_create, which runs after the object is saved but before the response is sent. Using a signal is also valid but can make the code flow less explicit.

69. Your API has a field that contains sensitive information (e.g., an API key for a third-party service). You want this field to be write-only so it is never included in any API responses. How do you achieve this?

- A) Use `write_only=True` on the field definition in the serializer.
- B) Override the `to_representation` method to remove the field.
- C) Use a `ReadOnlyField`.
- D) Both A and B are valid, but A is the standard and cleanest method.

Answer: A

Explanation: The `write_only=True` keyword argument is explicitly designed for this purpose. It ensures the field is used for validation and saving during create/update but is never included when serializing the object back to the client.

70. You want to implement "soft delete" (setting an `is_deleted` flag) instead of actual deletion for your model. How do you modify the destroy action in your `ModelViewSet`?

- A) Override the `perform_destroy` method to update the instance's `is_deleted` field and then save it, instead of calling `instance.delete()`.
- B) Override the `destroy` method and write the entire logic from scratch.
- C) You cannot do this with a `ModelViewSet`; you must use an `APIView`.
- D) Handle it in a pre-delete signal.

Answer: A

Explanation: `ModelViewSet` is designed to be customizable. The `destroy` method handles the response and HTTP status codes, while `perform_destroy(instance)` handles the actual deletion logic. Overriding `perform_destroy` is the cleanest way to change the deletion behavior without duplicating boilerplate code.

71. You need to add custom headers to every API response from your project. What is the best way to do this?

- A) Override the `dispatch` method in a base view class that all your views inherit from.
- B) Create a custom middleware class.
- C) Modify the `finalize_response` method in a custom renderer.
- D) Add the headers manually in every view.

Answer: B

Explanation: Middleware is the standard and most efficient Django component for applying cross-cutting concerns to all requests and responses, such as adding standard headers (e.g., `X-Frame-Options`, `Custom-API-Version`).

72. A client makes a request with an invalid token. What should your API return?

- A) A 401 Unauthorized response.
- B) A 403 Forbidden response.
- C) A 200 OK response with an error message in the body.
- D) A 404 Not Found response to hide the existence of the endpoint.

Answer: A

Explanation: HTTP 401 Unauthorized is specifically for authentication errors – the client needs to provide valid credentials (a valid token in this case). HTTP 403 Forbidden is for authorization – the client is authenticated but not permitted to perform the action.

73. You want to create a public, cached endpoint that lists non-sensitive data. The data changes only once per day. How can you improve performance?

- A) Use Django's cache framework to cache the entire view response.
- B) Use `prefetch_related` and `select_related` in `get_queryset`.
- C) Add a Cache-Control header to the response to instruct clients and proxies to cache it.
- D) All of the above.

Answer: D

Explanation: These are complementary strategies. A (view caching) saves server processing time. B (query optimization) makes the uncached request faster. C (HTTP caching) offloads traffic entirely by allowing clients and CDNs to store the response, which is highly effective for public, static data.

74. You are building a command-line tool that uses your API. The tool needs to handle paginated responses automatically and collect all results. What is a key property of the paginated response you should rely on?

- A) The count field, which tells the total number of results.
- B) The next field, which provides the URL to the next page of results.
- C) The results field, which contains the list for the current page.
- D) The previous field, which can be used to navigate backwards.

Answer: B

Explanation: The next field is the cornerstone of iterating through paginated results. The CLI tool should make a request, collect the results, then check if next is null. If it isn't, the tool should make a request to the URL in next and repeat the process until next is null, concatenating all the results.

75. You need to test that a specific permission class correctly denies access to a user. How would you write this test using APITestCase?

- A) Use `self.client.force_authenticate(user)` and then `self.assertRaises(PermissionDenied, self.client.get, url)`.
- B) Make the API request and assert that the `response.status_code` is 403.
- C) Mock the permission class and assert its `has_permission` method was called.
- D) Check the database to see if the request was blocked.

Answer: B

Explanation: Integration tests with `APITestCase` should assert on the HTTP response. The expected behavior when access is denied is to return a 403 Forbidden status code. You set up the scenario (authenticate a user who should be denied), make the request, and assert `response.status_code == status.HTTP_403_FORBIDDEN`.

Django REST Framework (DRF) - Scenario-Based MCQs (76-100)

76. You want to create a custom throttle that limits users based on the specific API plan they are subscribed to (e.g., Free: 100 req/hr, Pro: 1000 req/hr). How would you implement this?

- A) Create a custom throttle class that checks `request.user.profile.plan` and uses a different rate based on that value.
- B) Use `UserRateThrottle` and hope all users have the same rate.
- C) Use `ScopedRateThrottle` and set the scope based on the user's plan in the view.
- D) It's not possible to have dynamic throttle rates.

Answer: A

Explanation: Throttle rates are typically defined in settings. To make them dynamic, you need a custom throttle class. In its `allow_request` method, you would access `request.user.profile.plan`, and then determine the appropriate rate string (e.g., '100/hour') dynamically. You might need to override `get_cache_key` to include the plan in the key if rates differ per plan.

77. You have an endpoint that accepts a list of IDs to perform a bulk operation. What is a important security consideration for the `get_queryset` method when fetching these objects?

- A) There is no security consideration; `get_queryset` is not used for bulk operations.
- B) You must ensure `get_queryset` filters based on the user to prevent them from operating on objects they don't own, even if they know the IDs.
- C) You should disable pagination for bulk operations.
- D) You should use a separate, less secure queryset for performance.

Answer: B

Explanation: This is critical. Even in bulk operations, you must use `get_queryset()` to fetch the objects. This applies the standard filtering (e.g., `queryset.filter(owner=request.user)`) so that a user can only affect objects they are permitted to access. Failing to do this is a major security vulnerability.

78. You want to version your API by including the version in the Accept header: Accept: application/json; version=2.0. What versioning class do you use?

- A) `URLPathVersioning`
- B) `NamespaceVersioning`
- C) `AcceptHeaderVersioning`
- D) `HostNameVersioning`

Answer: C

Explanation: `AcceptHeaderVersioning` is designed to determine the API version from the Accept header, which is a common and clean versioning method that keeps URLs stable.

79. You need to integrate a complex, legacy authentication system that uses custom headers and cryptographic signatures. What is the best approach?

- A) Create a custom authentication class.
- B) Use `TokenAuthentication` and try to make the legacy system fit.
- C) Handle the authentication in middleware.
- D) Reject the requirement and ask for a simpler system.

Answer: A

Explanation: DRF's authentication system is pluggable for this reason. You would create a class that inherits from `BaseAuthentication`, implement the `authenticate` method to parse the custom headers and validate the signature, and return a (user, auth) tuple if successful.

80. You are seeing the "N+1 queries" problem in a nested serializer. You've added `prefetch_related` to the main view's `queryset`, but the problem persists. What is a likely cause?

- A) The database is too slow.
- B) You used `select_related` instead of `prefetch_related`.
- C) The nested serializer is itself accessing more related objects that are not prefetched.
- D) The problem is solved; your diagnosis must be wrong.

Answer: C

Explanation: This is a common pitfall. You might have prefetched `main_objects` and

their `related_set`, but if the serializer for each item in `related_set` then tries to access `related_set.another_related_field`, it will trigger a new query for each item. The solution is to extend your `prefetch_related` to cover the deeper relationship: `prefetch_related('related_set__another_related_field')`.

81. You want to create a custom action that is available on the list endpoint (e.g., POST `/api/users/bulk_create/`). What do you set `detail` to in the `@action` decorator?

- A) `detail=True`
- B) `detail=False`
- C) `detail=None`
- D) You don't use `@action` for list-level actions.

Answer: B

Explanation: The `detail` argument in the `@action` decorator specifies whether the action is intended for a single object (`detail=True`, URL includes a PK) or the entire collection (`detail=False`, base list URL).

82. Your API has a field that should be unique, but you want the validation error to be returned in the same structure as other field errors. How do you ensure this?

- A) You don't; Django will raise a `IntegrityError` which becomes a 500 error.
- B) Catch the `IntegrityError` in the view and return a 400 error.
- C) Use the `unique=True` validator on the model field and a `UniqueValidator` on the serializer field.
- D) Use a `SerializerMethodField` to manually check uniqueness.

Answer: C

Explanation: Using `UniqueValidator` on the serializer field is the correct approach. This validator runs during serializer validation (`is_valid()`), catches the issue early, and adds a clean validation error to the field's error list, resulting in a standard 400 response with structured errors. Letting it become an `IntegrityError` is a security and UX flaw.

83. You need to run a different set of throttles for a specific view. How do you do it?

- A) Set the `throttle_classes` attribute on the view.
- B) Modify the `DEFAULT_THROTTLE_CLASSES` setting for that view.
- C) Override the `get_throttles` method in the view.
- D) Both A and C are valid.

Answer: D

Explanation: You can either set the `throttle_classes` attribute to a fixed list or override

the `get_throttles` method for more dynamic control (e.g., based on action or request properties). Both are standard patterns in DRF.

84. A client needs to upload a very large file (several GBs). What is a concern with the standard DRF file upload handling?

- A) DRF cannot handle files larger than 1MB.
- B) The entire file will be loaded into memory, which can cause the server to crash.
- C) There is no concern; Django handles this efficiently by streaming to disk.
- D) The file will be uploaded but cannot be saved to a `FileField`.

Answer: B

Explanation: By default, Django (and by extension DRF) will read the entire uploaded file into memory before passing it to the view. For very large files, this is problematic. The solution is to use a library like `django-chunked-upload` or to write a custom view that handles the upload in chunks.

85. You want to create a custom renderer that outputs XML. What media type should you set?

- A) `'application/xml'`
- B) `'text/xml'`
- C) `'application/json'`
- D) Either A or B are acceptable.

Answer: D

Explanation: Both `'application/xml'` and `'text/xml'` are widely accepted MIME types for XML data. The IETF recommends `application/xml`, but `text/xml` is still very common. You could choose one or support both.

86. You are building a high-traffic API. What is one of the most impactful database-level optimizations you can make for read-heavy endpoints?

- A) Add more Python workers.
- B) Add database indexes on fields commonly used for filtering and ordering.
- C) Use `APITestCase` for testing.
- D) Use `PageNumberPagination` instead of `LimitOffsetPagination`.

Answer: B

Explanation: Without proper indexes, database queries will perform full table scans, which become exponentially slower as the table grows. Adding indexes on fields used

in `filter()`, `order_by()`, and `get()` lookups is the single most important backend optimization for read performance.

87. You want to implement "field-level permissions": a field should only be visible to users with a specific permission. How can you achieve this?

- A) Override the `to_representation` method in the serializer and conditionally remove the field based on `request.user.has_perm(...)`.
- B) This is not possible without a third-party package.
- C) Use a `SerializerMethodField` and check permissions inside the method.
- D) Both A and C are valid.

Answer: D

Explanation: A is the general approach for any field. C works well for `SerializerMethodFields` where the value is calculated anyway. You can access the request via `self.context['request']` inside the serializer to check the user's permissions.

88. You need to make an API call to an external service *during* the validation or saving of a serializer. What is a important architectural consideration?

- A) It's a bad idea; never call external services from a serializer.
- B) Wrap the call in a try-catch block and handle potential timeouts or errors gracefully.
- C) Make the call asynchronously so it doesn't block the request.
- D) Both B and C are important considerations.

Answer: D

Explanation: Calling external services introduces failure modes (timeouts, errors) and latency. You must handle these gracefully (B) to avoid giving the user a generic 500 error. For slow services, making an asynchronous call (C) is crucial to avoid blocking your application workers. However, this often requires a task queue and changes the flow (e.g., the external call happens *after* the initial response is sent).

89. You want to deprecate an API field but still accept it for a period of time for backward compatibility. How can you achieve this?

- A) Keep the field in the serializer and mark it as `required=False`.
- B) Use the `deprecated=True` argument on the field.
- C) Use two serializers: an old one and a new one, and use versioning to choose between them.
- D) Document the field as deprecated and hope clients stop using it.

Answer: A

Explanation: The common strategy is to mark the field

as required=False and read_only=True (so it's present in output but ignored on input). Then, in the validate method, you can log a warning if the deprecated field is used. The field can be removed entirely in a future API version.

90. You are using JWT. Where should the client securely store the received access and refresh tokens?

- A) In localStorage in the browser.
- B) In a JavaScript variable.
- C) In an HttpOnly cookie.
- D) In the database.

Answer: C

Explanation: Storing tokens in HttpOnly cookies is the most secure option for browser-based clients, as it mitigates the risk of XSS attacks stealing the tokens. localStorage is accessible by JavaScript and is vulnerable to XSS. The database is for server-side storage.

91. You want to create a custom permission that checks a condition against the request's data. Which method do you implement?

- A) has_permission(self, request, view)
- B) has_object_permission(self, request, view, obj)
- C) validate_permission(self, request, data)
- D) Permissions cannot access request data.

Answer: A

Explanation: has_permission runs before the serializer is validated, so it doesn't have access to validated_data. However, it *does* have access to request.data (the raw input data). You can implement logic here based on request.data, but be cautious as the data is not yet validated.

92. A PATCH request comes in to update a user's profile. The serializer has a field with a UniqueValidator. The client does not include this field in the PATCH request. What happens?

- A) The validator runs and fails because it can't find the value.
- B) The validator is skipped because the field was not provided.
- C) The field is set to None.
- D) A validation error is raised.

Answer: B

Explanation: For partial updates (PATCH), fields that are not provided are not validated. The UniqueValidator will only run if the field is included in the request data.

93. You need to create a complex, multi-step object creation process (a "wizard"). What is a good DRF strategy?

- A) Create a single massive endpoint that handles all steps.
- B) Create separate endpoints for each step, storing temporary state in the user's session or a temporary database model.
- C) Force the client to handle all logic and send the complete data in one request.
- D) This is not possible with DRF.

Answer: B

Explanation: Breaking the process into multiple RESTful resources is the most scalable and clear approach. Each step can be a separate API call that validates its part of the data and persists a "state" object. The final step commits the entire process.

94. You want to change the default "403 Forbidden" response body to a custom format. How do you do it?

- A) Override the `handle_exception` method in your view.
- B) Create a custom `permission_denied` method in your view.
- C) Change the `REST_FRAMEWORK['EXCEPTION_HANDLER']` setting.
- D) Both A and C are valid, as a `PermissionDenied` exception is handled by the exception handler.

Answer: D

Explanation: When a permission class denies access, it raises a `PermissionDenied` exception. This exception is caught by the view's `handle_exception` method and, ultimately, by the project's configured `EXCEPTION_HANDLER`. You can customize the response format at either level.

95. You are building a real-time API that uses WebSockets. Can you use DRF permissions and authentication?

- A) No, WebSockets are completely separate from HTTP.
- B) Yes, you can re-use the same permission classes without modification.
- C) Yes, but you may need to write adapters to extract authentication details from the WebSocket connection handshake.
- D) Only if you use Django Channels.

Answer: C

Explanation: The concepts are reusable, but the mechanism is different. You would typically extract the token or session ID from the WebSocket handshake request headers (e.g., `sec-websocket-protocol` or query string), authenticate the user, and then apply permission logic based on that user within your WebSocket consumer. Django Channels helps with this.

96. You want to ensure a value is provided for a field *only* on creation, not on update. How can you do this?

- A) Use `required=True` and it will work automatically.
- B) Override the `__init__` method of the serializer to set the field's `required` attribute to `True` if the instance is `None` (create), and `False` otherwise (update).
- C) Use a custom `CreateOnlyDefault` validator.
- D) Both B and C are valid strategies.

Answer: D

Explanation: B is a common and effective pattern. DRF also provides a `CreateOnlyDefault` class that can be used as a default argument for fields that should only be used on create.

97. You need to integrate with a frontend that expects error responses in a different format than DRF's default. What is the most sustainable approach?

- A) Ask the frontend team to change their code.
- B) Manually reformat errors in every view.
- C) Create a custom exception handler and set it as the default.
- D) Fork and modify DRF.

Answer: C

Explanation: The custom exception handler is the designated, sustainable, and global solution for changing the format of all error responses (including validation errors, permission errors, etc.) across your entire API.

98. You want to add a `last_modified` field to your model that is automatically updated *only* when certain other fields change. How would you achieve this?

- A) Override the model's `save` method.
- B) Use a `pre_save` signal.
- C) Use a database trigger.
- D) Any of the above are valid.

Answer: D

Explanation: All are valid strategies. Overriding `save` keeps the logic with the model. A signal decouples the logic. A database trigger is the most efficient and robust as it works at the database level, even if the model is updated through raw SQL.

99. Your API has a GET endpoint that performs a very expensive calculation. The result is valid for 5 minutes. How can you prevent clients from overwhelming this endpoint?

- A) Use a very strict throttle.
- B) Implement server-side caching of the result for 5 minutes.
- C) Add a Cache-Control: max-age=300 header to the response.
- D) Both B and C are excellent complementary strategies.

Answer: D

Explanation: B (server-side caching) ensures the expensive calculation is only done once every 5 minutes, drastically reducing server load. C (HTTP caching) allows clients and proxies to cache the response, further reducing the number of requests that even reach your server. They work beautifully together.

100. You are finalizing your API for version 1.0. What is a crucial practice to establish before release?

- A) Freeze the API schema and communicate any future changes through versioning.
- B) Promise to never change anything.
- C) Use APIView for everything to ensure flexibility.
- D) Don't document it to maintain flexibility.

Answer: A

Explanation: For API consumers, stability is paramount. Once you have users, you must version your API. Changes and new features should be added in a new version (e.g., v1.1, v2.0), while the existing v1.0 API is maintained and eventually deprecated on a published schedule. This allows clients to migrate at their own pace.